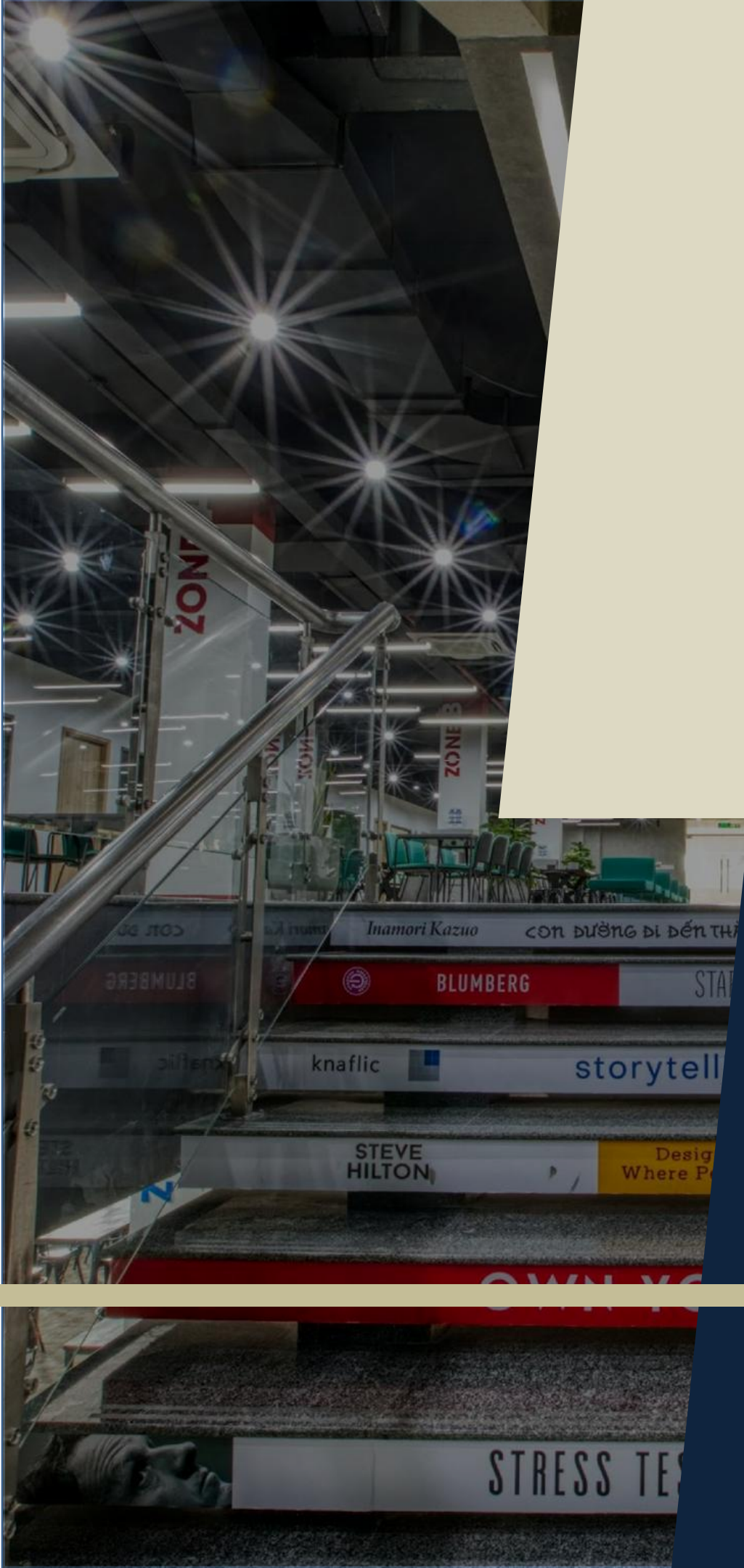


# Coding Practice

## Design patterns



The ratio of time spent reading code to writing code is 10:1 so it is important to write code that is as readable as possible.

~Robert C. Martin~

Even a fool can write code that a computer can understand.  
Good programmers write code that humans can understand.

~Martin Fowler~

*Don't reinvent the wheel !!!*

# Learn programming languages

## Keywords

- Variable
- Function
- Class
- Object
- Expression
- Operator
- ...

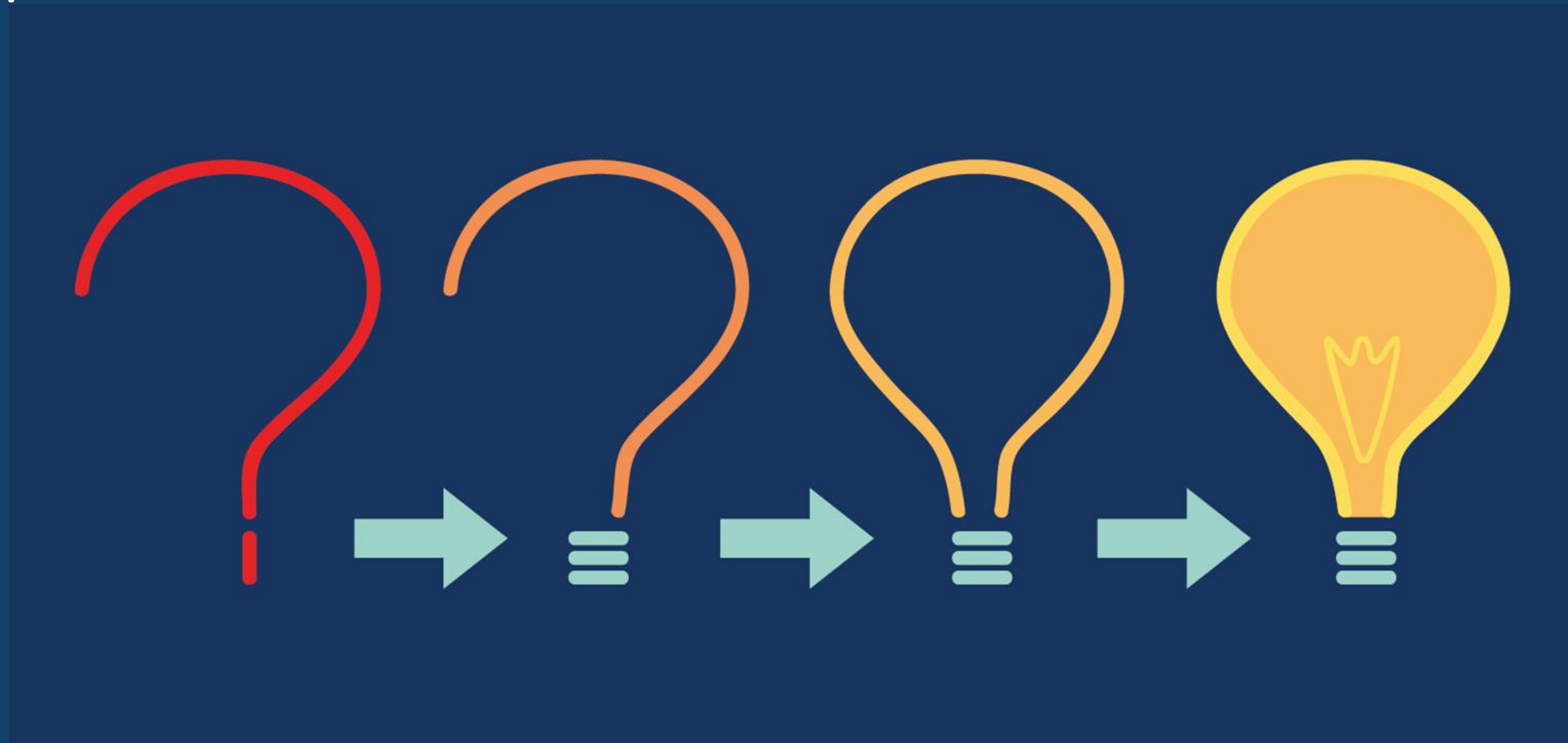
## Syntax

- Command
- Condition
- Loop
- Class declaration
- Call method
- Identifier
- ...

## Style

- Convention
- Principle
- Pattern
- ...

What is design ?



Analysis and problem solving

## What is a good design ?

- Not only solves **current problems** But also solve the **problem in the future**.
- Not only does it run well when the **system is small** But also runs well when the **system is larger**
- Not only runs well with **current version** But **more versions** can be well.

## Design Principles

- Encapsulate what varies
- Favor composition over inheritance
- Programming to interfaces not to implementations
- Strive for loosely coupled between objects that interact
- Classes should OPEN for extension and CLOSE for modification
- Depends on abstraction, not concrete classes
- Don't call us, we'll call you
- A class should have only one reason to change

## Template



## Patterns are everywhere



Patterns are everywhere

Google search results for "design pattern".

**Refactoring.Guru**  
https://refactoring.guru/design-patterns · Dịch trang này ·  
**Design Patterns**  
Design Patterns are typical solutions to commonly occurring problems in software design. They are blueprints that you can customize to solve a particular ...  
Catalog · Code Examples · What is a Pattern · Creational Patterns

**Viblo**  
https://viblo.asia/design-patterns-la-gi-tai-sao-no-lai-la... ·  
**Design Patterns là gì? Tổng hợp 23 mẫu Design Pattern**  
Design pattern là các giải pháp tổng thể đã được tối ưu hóa, được tái sử dụng cho các vấn đề phổ biến trong thiết kế phần mềm mà chúng ta thường gặp phải hàng ...  
Interpreter Design Pattern · Tổng quan về Singleton... · Factory Method · Adapter

**TopDev**  
https://topdev.vn/blog/design-pattern-la-gi ·  
**Design pattern là gì? Tại sao nên sử dụng Design pattern?**  
Design pattern là các giải pháp tổng thể đã được tối ưu hóa, được tái sử dụng cho các vấn đề phổ biến trong thiết kế phần mềm mà chúng ta thường gặp phải ...

**SourceMaking**  
https://sourcemaking.com/design\_patt... · Dịch trang này ·  
**Design Patterns**  
A design pattern is a general repeatable solution to a commonly occurring problem in software design.

**Viblo**  
https://viblo.asia/tong-hop-cac-bai-huong-dan-ve-desi... ·  
**Tổng Hợp Hướng Dẫn 23 Design Pattern Cơ Bản Của GoF**  
Design Pattern là một kỹ thuật trong lập trình hướng đối tượng, nó khá quan trọng và

**Mẫu thiết kế phần mềm (Software design pattern)**

Creational: 1. Singleton, 2. Factory, 3. Abstract Factory, 4. Builder, 5. Prototype

Structural: 6. Adapter, 7. Composite, 8. Proxy, 9. Flyweight, 10. Decorator

Behavioural: 11. Strategy, 12. Command, 13. Template Method, 14. Mediator, 15. Chain of Responsibility, 16. Observer, 17. State

Trong công nghệ phần mềm, một mẫu thiết kế là một giải pháp tổng thể cho các vấn đề chung trong thiết kế phần mềm. Wikipedia >

Mọi người cũng tìm kiếm

Thiết kế, Trí tuệ nhân tạo, Lập trình hướng đối tượng, Lập trình máy tính

Ý kiến phản hồi

## What is Design Pattern ?

- Is a reusable general solution to common problems
- Is a model or description of how to solve a problem
- A proven technique
- Design Pattern is not a complete design that can be directly transformed into source code.
- Design Patterns are templates for solving problems in different situations.

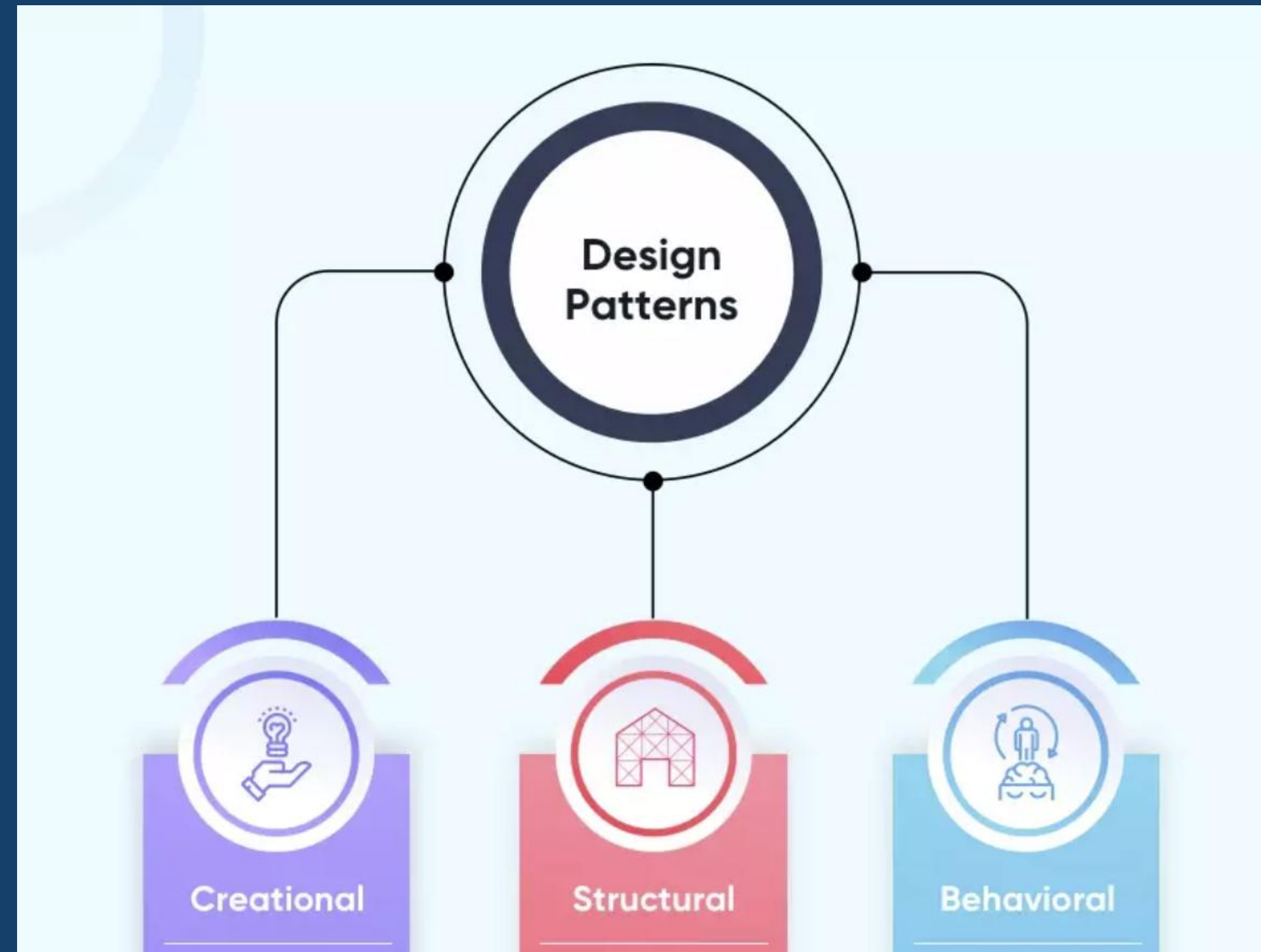
## Why do we need Design Pattern?

- Speed up software design and development
- Proven quality of solution
- Prevent problems that arise if the design is not good
- Can be applied to many different situations
- Easily collaborate, share designs and source code between parties

## Classification of Design Patterns

Design patterns are classified into three main categories:

- **Creational Patterns:** Deal with object creation mechanisms.
- **Structural Patterns:** Focus on object composition and relationships.
- **Behavioral Patterns:** Concerned with communication between objects.



# Overview of Creational Patterns

Creational patterns deal with object creation mechanisms, optimizing how objects are created to be flexible and reusable.

Common Creational Patterns:

1. Singleton Pattern
2. Factory Method Pattern
3. Abstract Factory Pattern
4. Builder Pattern
5. Prototype Pattern

## Singleton Pattern

Purpose: Ensures that a class has only one instance and provides a global point of access to it.

Example: Managing a single database connection in an application.

# Singleton Pattern

```
1  ✓ public class Singleton
2      {
3          // Static field to hold the single instance of the class
3 references
4          private static Singleton _instance;
5
6          // Object used for locking to ensure thread safety
1 reference
7          private static readonly object _lock = new object();
8
9          // Private constructor to prevent instantiation from outside the class
1 reference
10         private Singleton() { }
11
12         // Static method to provide a global access point to the single instance
0 references
13  ✓ public static Singleton GetInstance()
14      {
15          // Lock ensures only one thread can execute this block at a time
16  ✓ lock (_lock)
17      {
18          // Check if the instance has already been created
19          if (_instance == null)
20              _instance = new Singleton(); // Create the instance if it doesn't exist
21          return _instance; // Return the single instance
22      }
23      }
24  }
```

## Factory Method Pattern

**Purpose:** Provides an interface for creating objects, but allows subclasses to alter the type of objects created.

**Example:** Creating different types of shapes (circle, square) without specifying their exact classes.

```
// Common interface for shape
2 references
public interface IShape
{
    2 references
    void Draw();
}

// Circle
0 references
public class Circle : IShape
{
    1 reference
    public void Draw()
    {
        Console.WriteLine("Drawing a Circle");
    }
}
```

```
// Square
0 references
public class Square : IShape
{
    1 reference
    public void Draw()
    {
        Console.WriteLine("Drawing a Square");
    }
}
```

Structural patterns deal with how classes and objects are composed to form larger structures.

- **Common Structural Patterns:**

- Adapter Pattern
- Decorator Pattern
- Facade Pattern
- Composite Pattern
- Proxy Pattern

## Adapter Pattern

Purpose: Converts the interface of a class into another interface that clients expect.

Example: Adapting legacy code to work with a new system.

```
public interface ITarget
{
    1 reference
    void Request();
}
2 references
public class Adaptee
{
    1 reference
    public void SpecificRequest()
    => Console.WriteLine("Called SpecificRequest");
}
```

```
public class Adapter : ITarget
{
    2 references
    private readonly Adaptee _adaptee;

    0 references
    public Adapter(Adaptee adaptee)
    {
        _adaptee = adaptee;
    }

    1 reference
    public void Request()
    {
        _adaptee.SpecificRequest();
    }
}
```

## Decorator Pattern

**Purpose:** Allows behavior to be added to an individual object dynamically without affecting the behavior of other objects from the same class.

**Example:** Adding new functionalities to a window (scrolling, borders).

Example: [Click Here](#)

## Overview of Behavioral Patterns

Behavioral patterns focus on communication between objects and how responsibilities are distributed.

- **Common Behavioral Patterns:**

- Observer Pattern
- Strategy Pattern
- Command Pattern
- State Pattern
- Mediator Pattern

## Observer Pattern

**Purpose:** Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified.

**Example:** Event handling in a GUI application.

## Command Pattern

**Purpose:** Encapsulates a request as an object, thereby allowing users to parameterize clients with different requests, queue or log requests, and support undoable operations.

Example: [Click here](#)

## **1. Design Patterns: Elements of Reusable Object-Oriented Software**

**Author:** Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides (Nhóm "Gang of Four")

## **2. Head First Design Patterns**

**Author:** Eric Freeman, Elisabeth Robson

## **3. Patterns of Enterprise Application Architecture**

**Author:** Martin Fowler

## **4. Refactoring to Patterns**

**Author:** Joshua Kerievsky

## **5. Clean Architecture: A Craftsman's Guide to Software Structure and Design**

**Author:** Robert C. Martin (Uncle Bob)

## **6. Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design**

**Author:** Craig Larman

## **7. Pro Design Patterns in C#**

**Author:** Chaur Wu

*Start your future at EIU*

---

**Q&A**

*Start your future at EIU*

---

**THANK YOU**